5 Enviar e acompanhar

CURSO PARA INTEGRADORES DO HUBSAÚDE · MÓDULO 5 DE 6

Com o registro montado no módulo 3 e o token obtido no módulo 4, o seu sistema está pronto para enviar. Neste módulo, você envia o registro ao Simulador, acompanha o processamento até o resultado e aprende a tratar recusas e quedas de conexão sem duplicar registros.

AO FINAL, VOCÊ SERÁ CAPAZ DE

- ✓ enviar um registro com os cabeçalhos obrigatórios;
- ✓ explicar o envio em duas etapas e acompanhar o processamento pela Task;
- ✓ interpretar um registro aceito e um registro recusado;
- ✓ repetir um envio com segurança usando a `Idempotency-Key`;
- ✓ realizar o envio completo a partir de um programa em Java.

COMO O MÓDULO ESTÁ ORGANIZADO

1. **Conceitos:** seções 5.1 a 5.4
2. **Prática guiada:** 6 passos no terminal
3. **Verificação:** perguntas, glossário e respostas

VOCÊ VAI PRECISAR

- O Simulador em funcionamento, o sistema cadastrado e os certificados, como no módulo 2.
- A variável `TOKEN`, criada no passo 3 do módulo 4, na mesma janela de terminal.
- Os arquivos `paciente.json`, corrigido no módulo 3, e `paciente-com-erros.json`.
- Para o passo 6: o projeto Java do módulo 4.

ASSISTA AOS VÍDEOS

Aponte a câmera do celular para o código.

SEÇÕES 5.1 A 5.4
O envio em duas etapas
5 min 27 s, com narração e legendas

PASSOS 1 A 5
Envio e acompanhamento pelo terminal
6 min 25 s, com narração e legendas

DÚVIDAS E SUGESTÕES

Se algum passo ficar difícil, escreva para o suporte do HubSaúde, em `suporte@saude.go.gov.br`, com o assunto "Curso para integradores, módulo 5". A versão mais recente deste módulo, com os vídeos, está em `https://sesgo-ti.github.io/hubsaude/curso/enviar/`.

Copiar de um PDF pode alterar aspas, hífens e quebras de linha. Para os comandos e arquivos, prefira a versão do módulo no portal ou os arquivos para baixar, em `https://sesgo-ti.github.io/hubsaude/curso/materiais/`.

PARTE 1

Conceitos

5.1 Por que o envio tem duas etapas

Em muitas APIs, a resposta ao envio já traz o resultado: o sistema envia e espera, com a conexão aberta, até o servidor terminar de conferir o conteúdo. No HubSaúde, o recebimento é confirmado na hora, e o conteúdo é processado depois.

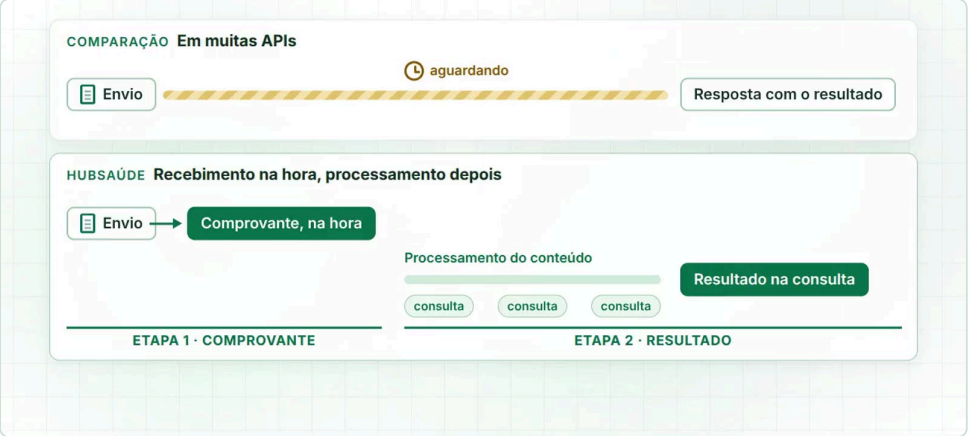


Figura 5.1. Acima, o modelo comum, em que o sistema espera o resultado. Abaixo, o HubSaúde: o comprovante chega na hora, e o resultado é obtido por consulta. Quadro do vídeo "O envio em duas etapas".

Etapa 1: comprovante

O sistema envia o registro com `POST /fhir/{tipo}`. A resposta é 202, sem corpo, com o endereço de uma Task: o equivalente a dizer "recebi e vou processar".

Etapa 2: resultado

O sistema consulta a Task nesse endereço até o processamento terminar. A Task informa se o registro foi aceito ou recusado e, se recusado, por quê.

Pense em um protocolo de atendimento: ao entregar um documento, você recebe um número e, depois, consulta o andamento por ele. Assim, o seu sistema não fica parado esperando enquanto o conteúdo é conferido.

5.2 Os cinco cabeçalhos obrigatórios

Todo envio leva o registro no corpo e cinco cabeçalhos:

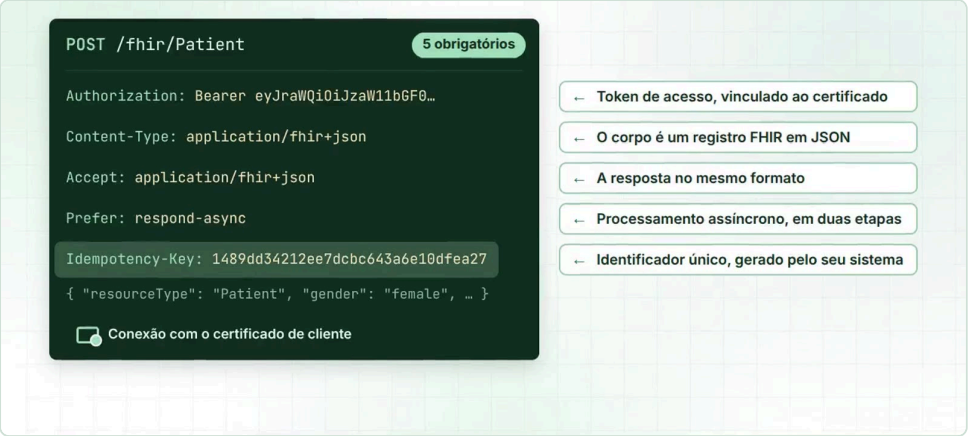


Figura 5.2. O pedido de envio, com os cinco cabeçalhos e o papel de cada um. Quadro do vídeo "O envio em duas etapas".

Cabeçalho	Valor	Para que serve
`Authorization`	`Bearer` seguido do token	A autorização obtida no módulo 4.
`Content-Type`	`application/fhir+json`	Informa que o corpo é um registro FHIR em JSON.
`Accept`	`application/fhir+json`	Pede a resposta no mesmo formato.
`Prefer`	`respond-async`	Pede o processamento em duas etapas.
`Idempotency-Key`	Um identificador único, como um UUID	Permite repetir o envio sem duplicar o registro (seção 5.4).

A falta de qualquer um deles resulta em 400, com um OperationOutcome que aponta o problema.

> **ATENÇÃO**
> Como o token é vinculado ao certificado, toda chamada, inclusive o envio e cada consulta à Task, precisa apresentar o **certificado de cliente**.

5.3 O comprovante e a Task

A resposta 202 traz dois cabeçalhos importantes: o `Content-Location`, com o endereço da Task, e o `Retry-After`, que sugere, em segundos, quanto tempo esperar antes da consulta. A Task é um recurso FHIR próprio para acompanhar pedidos em andamento, e o campo `status` diz em que ponto o processamento está:

status	Significado	O que o sistema faz
`accepted` ou `in-progress`	O processamento ainda não terminou.	Aguarda e consulta de novo.
`completed`	O registro foi aceito.	Lê em `output` o endereço do registro criado.
`rejected`	O registro foi recusado.	Lê o resumo em `statusReason` e os problemas no OperationOutcome, em `contained`.

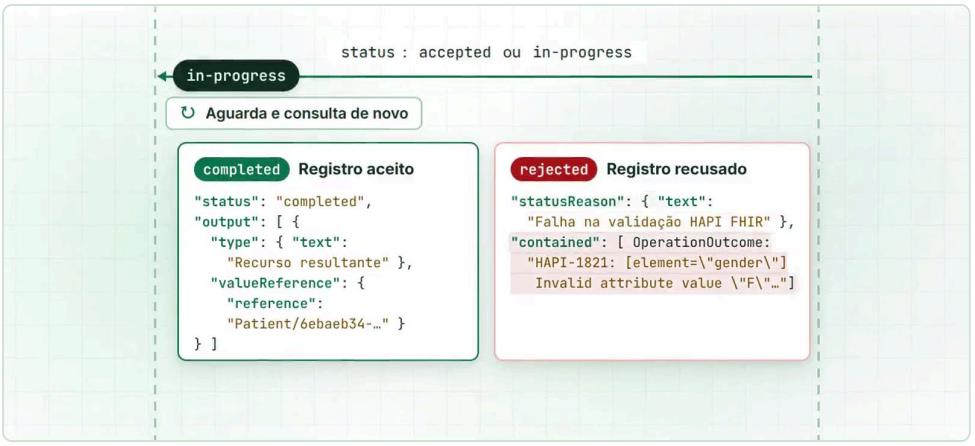


Figura 5.3. Enquanto o status é `in-progress`, a consulta se repete. Ao final, `completed` aponta o registro criado, e `rejected` traz o motivo. Quadro do vídeo "O envio em duas etapas".

5.4 Repetir sem duplicar

Imagine que a conexão caia logo depois do envio, e o seu sistema não saiba se o registro chegou. Se enviar de novo com uma chave nova, o HubSaúde trata o envio como outro registro, e a informação pode ficar duplicada. A solução é repetir o envio com a **mesma** `Idempotency-Key`: o HubSaúde reconhece a repetição.



Figura 5.4. As duas respostas possíveis a uma repetição com a mesma chave. Quadro do vídeo "O envio em duas etapas".

Quando a repetição chega	Resposta	O que fazer
Depois do fim do processamento	202, com o mesmo endereço de Task e o cabeçalho `Idempotency-Replayed: true`	Consultar a Task. Nenhum registro novo foi criado.
Durante o processamento do original	409, com o mesmo endereço de Task e um `Retry-After`	Aguardar e repetir com a mesma chave, ou consultar a Task.

ATENÇÃO

Gere a chave **uma vez por registro** e guarde-a até a confirmação do resultado. Uma chave nova a cada tentativa faz o HubSaúde tratar a repetição como um envio diferente.

EM RESUMO

- O envio tem duas etapas: o 202 confirma apenas o recebimento, e o resultado está na Task.
- Todo envio leva cinco cabeçalhos obrigatórios e o certificado de cliente.
- A Task termina em `completed`, com o registro criado em `output`, ou em `rejected`, com o motivo em `contained`.
- A mesma `Idempotency-Key` permite repetir um envio sem duplicar o registro.

PARTE 2

Prática guiada

Execute os comandos a partir da pasta `curso-hubsaude`, na mesma janela de terminal em que a variável `TOKEN` foi criada no módulo 4. Se o token tiver expirado, repita os passos 1 e 3 daquele módulo. Para encurtar os comandos, defina antes a variável que reúne as opções de certificado:

VOCÊ DIGITA

```
$ CERTIFICADOS="--cacert simulador-server.pem --cert client-cert.pem --key client-key.pem"
```

1 Enviar o registro

Enviar o paciente corrigido no módulo 3 e guardar o comprovante.

Primeiro, gere a `Idempotency-Key` e guarde-a na variável `CHAVE`. Depois, envie o arquivo com os cinco cabeçalhos:

VOCÊ DIGITA

```
$ CHAVE=$(openssl rand -hex 16)
$ curl -s $CERTIFICADOS -X POST https://localhost:8443/fhir/Patient \
    -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/fhir+json" \
    -H "Accept: application/fhir+json" \
    -H "Prefer: respond-async" \
    -H "Idempotency-Key: $CHAVE" \
    --data @paciente.json \
    -D cabecalhos.txt -o /dev/null -w '%{http_code}\n'
```

Parte do comando	O que faz
`--data @paciente.json`	Envia o conteúdo do arquivo como corpo.
`-D cabecalhos.txt`	Grava os cabeçalhos da resposta no arquivo.
`-o /dev/null`	Descarta o corpo da resposta, que no 202 é vazio.
`-w '%{http_code}\n'`	Exibe apenas o código da resposta.

VOCÊ VÊ

```
202
```

Guarde o endereço da Task na variável `TASK` e veja os cabeçalhos:

VOCÊ DIGITA

```
$ TASK=$(grep -i '^content-location' cabecalhos.txt | cut -d' ' -f2 | tr -d '\r')
$ cat cabecalhos.txt
```

VOCÊ VÊ

```
HTTP/1.1 202
X-Content-Type-Options: nosniff
Content-Location: https://localhost:8443/Task/514a9c8e-3d23-491e-835b-129782e06407
Retry-After: 30
Preference-Applied: respond-async
traceparent: 00-2a4411a585c54ba3a5ef51580b01e794-b3c22a12e8634c69-01
correlation-id: 2a4411a585c54ba3a5ef51580b01e794
Content-Length: 0
Date: Fri, 30 Oct 2026 23:04:37 GMT
```

OBSERVE

O `Content-Location` traz o endereço da Task, e o `Retry-After` sugere esperar 30 segundos. O `correlation-id` identifica o pedido: informe-o ao suporte se precisar de ajuda com um envio.

☐ **Passo concluído**

2 Acompanhar o processamento

Consultar a Task até o resultado e ler o registro criado.

VOCÊ DIGITA

```
$ curl -s $CERTIFICADOS "$TASK" -H "Authorization: Bearer $TOKEN" | jq '{status, output}'
```

Logo depois do envio, o processamento ainda não terminou:

VOCÊ VÊ

```
{
  "status": "in-progress",
  "output": null
}
```

Repita o comando alguns segundos depois:

VOCÊ VÊ

```
{
  "status": "completed",
  "output": [
    {
      "type": {
        "text": "Recurso resultante"
      },
      "valueReference": {
        "reference": "Patient/8b10ac07-4155-4985-b263-702b92051262"
      }
    }
  ]
}
```

O `output` aponta o paciente criado. Leia o registro por esse endereço:

VOCÊ DIGITA

```
$ REGISTRO=$(curl -s $CERTIFICADOS "$TASK" -H "Authorization: Bearer $TOKEN" | jq -r
  '.output[0].valueReference.reference')
$ curl -s $CERTIFICADOS "https://localhost:8443/fhir/$REGISTRO" -H "Authorization: Bearer $TOKEN" -H
  "Accept: application/fhir+json"
```

VOCÊ VÊ

```
{
  "resourceType": "Patient",
  "id": "8b10ac07-4155-4985-b263-702b92051262",
  "name": [ {
    "use": "official",
    "text": "Maria da Silva"
  } ],
  "gender": "female",
  "birthDate": "1990-01-01"
}
```

OBSERVE

O registro é o mesmo do arquivo, acrescido do `id` atribuído pelo servidor. Os identificadores no seu computador serão outros. A leitura usa o escopo `system/Patient.rs`, concedido no módulo 2.

☐ **Passo concluído**

---

3 Repetir o envio com a mesma chave

Simular uma queda de conexão e confirmar que o registro não é duplicado.

Execute novamente apenas o `curl` do passo 1, **sem** gerar uma nova `CHAVE`. O código exibido é de novo 202. Veja os cabeçalhos:

VOCÊ DIGITA

```
$ cat cabecalhos.txt
```

VOCÊ VÊ

```
HTTP/1.1 202
X-Content-Type-Options: nosniff
Content-Location: https://localhost:8443/Task/514a9c8e-3d23-491e-835b-129782e06407
Retry-After: 30
Preference-Applied: respond-async
Idempotency-Replayed: true
traceparent: 00-69e6e266661148ce9175de88cdea112b-81b324cae6bb475a-01
correlation-id: 69e6e266661148ce9175de88cdea112b
Content-Length: 0
Date: Fri, 30 Oct 2026 23:04:41 GMT
```

OBSERVE

O endereço da Task é o mesmo do passo 1, e o cabeçalho `Idempotency-Replayed: true` confirma que o HubSaúde reconheceu a repetição. Nenhum novo paciente foi criado.

E se a repetição chegar durante o processamento?

Gere uma nova chave e execute o `curl` do passo 1 duas vezes seguidas, sem esperar entre uma execução e outra:

VOCÊ DIGITA

```
$ CHAVE=$(openssl rand -hex 16)
```

O primeiro envio recebe 202. O segundo chega enquanto o primeiro ainda está em processamento e recebe 409. Os cabeçalhos da segunda resposta:

VOCÊ VÊ

```
HTTP/1.1 409
X-Content-Type-Options: nosniff
Content-Location: https://localhost:8443/Task/acb2ef23-9538-4080-b826-661efef5f60d
Retry-After: 5
traceparent: 00-8ab62d78a3e54f6184b4a44230ea518c-68ff359aeaaa4a9e-01
correlation-id: 8ab62d78a3e54f6184b4a44230ea518c
Content-Type: application/fhir+json
Content-Length: 282
Date: Wed, 04 Nov 2026 15:05:15 GMT
```

OBSERVE

O `Content-Location` aponta a mesma Task do primeiro envio, e o `Retry-After` sugere aguardar 5 segundos. O corpo é um OperationOutcome com o código `duplicate`. O 409 não é uma recusa do registro: aguarde e repita com a mesma `CHAVE`, ou consulte a Task.

☐ **Passo concluído**

---

4 Enviar um registro inválido

Enviar o registro com erros e ler o motivo da recusa.

Salve a versão com erros do módulo 3 como `paciente-com-erros.json`, ou baixe-a pronta:

paciente-com-erros.json

```
{
  "resourceType": "Patient",
  "name": [{"use": "official", "text": "Maria da Silva"}],
  "gender": "F",
  "birthDate": "01/01/1990"
}
```

Para baixar: `https://sesgo-ti.github.io/hubsaude/curso/arquivos/paciente-com-erros.json`

Envie-a com uma nova chave, guarde o endereço da Task e consulte-a:

VOCÊ DIGITA

```
$ CHAVE=$(openssl rand -hex 16)
$ curl -s $CERTIFICADOS -X POST https://localhost:8443/fhir/Patient \
    -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/fhir+json" \
    -H "Accept: application/fhir+json" \
    -H "Prefer: respond-async" \
    -H "Idempotency-Key: $CHAVE" \
    --data @paciente-com-erros.json \
    -D cabecalhos.txt -o /dev/null -w '%{http_code}\n'
$ TASK=$(grep -i '^content-location' cabecalhos.txt | cut -d' ' -f2 | tr -d '\r')
$ curl -s $CERTIFICADOS "$TASK" -H "Authorization: Bearer $TOKEN" | jq '{status, statusReason,
  contained}'
```

O envio é aceito com 202: o comprovante não depende do conteúdo. Repita a última linha até o fim do processamento:

VOCÊ VÊ


```
{
  "status": "rejected",
  "statusReason": {
    "text": "Falha na validação HAPI FHIR"
  },
  "contained": [
    {
      "resourceType": "OperationOutcome",
      "id": "outcome",
      "issue": [
        {
          "severity": "error",
          "code": "invalid",
          "diagnostics": "HAPI-1821: [element=\"gender\"] Invalid attribute value \"F\":
            Unknown AdministrativeGender code 'F'"
        }
      ]
    }
  ]
}
```


DESAFIO

Compare com o resultado do Validador no módulo 3. Qual erro o Simulador deixou de apontar?

OBSERVE

O Simulador apontou apenas o problema de `gender`; o Validador apontou também a data. O Simulador confere somente a estrutura do registro, enquanto o ambiente oficial aplica as regras completas dos Guias de Implementação. Por isso, valide sempre os registros antes de enviá-los.

☐ **Passo concluído**

5 Provocar falhas de requisição

Reconhecer as respostas quando falta um cabeçalho, o certificado ou a permissão.

Diferentemente da recusa do passo 4, estas falhas acontecem já na primeira etapa: o envio nem chega a receber um comprovante. Repita o envio do passo 1 em cada situação:

Situação	Como provocar	Código	Por quê
Falta um cabeçalho	Retire a linha da `Idempotency-Key`.	400	O cabeçalho é obrigatório; o OperationOutcome diz qual falta.
Falta o certificado	Troque `$CERTIFICADOS` por `--cacert simulador-server.pem`.	401	O token é vinculado ao certificado, e a chamada não o apresentou.
Falta a permissão	Obtenha um token, como no módulo 4, com `-d "scope=system/Patient.rs system/Task.rs"`, e envie com ele.	403	O token não inclui `system/Patient.c`, necessário para criar.

DICA

Uma forma de memorizar: 400 é um problema no pedido; 401, na identidade; 403, na permissão. Nos três casos, corrija a causa antes de repetir: insistir não muda o resultado.

☐ Passo concluído

6 Enviar a partir de um programa

Combinar a obtenção do token, o envio e o acompanhamento em um único programa Java.

O programa amplia o do módulo 4. A biblioteca oficial cuida apenas do token; as chamadas de envio e de consulta são feitas pelo seu próprio código, com o cliente HTTP do Java. No projeto do módulo 4, copie para a pasta `java` os arquivos `paciente.json` e `paciente-com-erros.json` e salve o programa como `src/main/java/EnviarPaciente.java`. Ele é apresentado em três partes.

Parte 1: o token e a conexão

`java/EnviarPaciente.java · linhas 21 a 42`

```
21  public static void main(String[] args) throws Exception {
22      String arquivo = args.length > 0 ? args[0] : "paciente.json"; (1)
23
24      try (var client = SmartTokenClient.builder() (2)
25              .fhirBase(SIMULADOR)
26              .clientId("meu-sistema")
27              .privateKeyPem(Path.of("client-key.pem"))
28              .certificatePem(Path.of("client-cert.pem"))
29              .serverTrustAnchor(Path.of("simulador-server.pem"))
30              .build()) {
31
32          String token = client.obtainToken("system/Patient.c system/Patient.rs system/Task.rs"); (3)
33
34          SSLContext conexao = SslContextFactory.buildSslContext( (4)
35                  Path.of("simulador-server.pem"),
36                  SslContextFactory.DEFAULT_TLS_PROTOCOL,
37                  PemLoader.loadPrivateKey(Path.of("client-key.pem")),
38                  PemLoader.loadCertificate(Path.of("client-cert.pem")));
```

```
39
40          var mapper = new ObjectMapper();
41
42          try (var http = HttpClient.newBuilder().sslContext(conexao).build()) {  5
```

1. O arquivo a enviar: `paciente.json`, ou o informado na execução.
2. O mesmo cliente da biblioteca do módulo 4.
3. O token, com os três escopos do curso.
4. A conexão mTLS, montada com a mesma chave e o mesmo certificado.
5. O cliente HTTP do Java, configurado com essa conexão.

Parte 2: o envio e o comprovante

java/EnviarPaciente.java · linhas 43 a 58

```
43  var envio = HttpRequest.newBuilder(URI.create(SIMULADOR + "/fhir/Patient"))  1
44          .header("Authorization", "Bearer " + token)  2
45          .header("Content-Type", "application/fhir+json")
46          .header("Accept", "application/fhir+json")
47          .header("Prefer", "respond-async")
48          .header("Idempotency-Key", UUID.randomUUID().toString())  3
49          .POST(HttpRequest.BodyPublishers.ofFile(Path.of(arquivo)))
50          .build();
51
52  var resposta = http.send(envio, HttpResponse.BodyHandlers.ofString());
53  if (resposta.statusCode() ≠ 202) {  4
54      throw new IllegalStateException(
55              "Envio recusado: " + resposta.statusCode() + " " + resposta.body());
56  }
57  String enderecoTask = resposta.headers().firstValue("Content-Location").orElseThrow();  5
58  System.out.println("Comprovante: " + enderecoTask);
```

1. O `POST` em `/fhir/Patient`.
2. Os cinco cabeçalhos obrigatórios, começando pelo token.
3. Uma `Idempotency-Key` nova, gerada como UUID.
4. Qualquer código diferente de 202 interrompe o programa.
5. O endereço da Task, lido do `Content-Location`.

Parte 3: o acompanhamento e o resultado

java/EnviarPaciente.java · linhas 60 a 84

```
60  JsonNode task;
61  String status;
62  do {  1
63      Thread.sleep(2000);  2
64      var consulta = HttpRequest.newBuilder(URI.create(enderecoTask))
65              .header("Authorization", "Bearer " + token)
66              .header("Accept", "application/fhir+json")
67              .GET()
68              .build();
69      task = mapper.readTree(http.send(consulta, HttpResponse.BodyHandlers.ofString()).body());
70      status = task.path("status").asString();  3
71  } while (status.equals("accepted") || status.equals("in-progress"));
72
```

```
if (status.equals("completed")) {                                                    4
    System.out.println("Registro aceito: "
            + task.path("output").path(0).path("valueReference").path("reference").asString());
} else {
    System.out.println("Registro recusado: " + task.path("statusReason").path("text").asString());  5
    for (JsonNode recurso : task.path("contained")) {
        for (JsonNode problema : recurso.path("issue")) {
            System.out.println("  " + problema.path("severity").asString()
                    + " " + problema.path("diagnostics").asString());
        }
    }
}
```

1. Repete a consulta enquanto o processamento não termina.
2. Espera dois segundos entre as consultas.
3. Lê o `status` da Task.
4. Aceito: exibe o endereço do registro criado.
5. Recusado: exibe o resumo e cada problema do OperationOutcome.

ATENÇÃO

Para o exercício, o programa gera uma `Idempotency-Key` a cada execução. Em um sistema real, a chave deve ser gerada uma vez por registro e guardada com ele até a confirmação do resultado, como na seção 5.4.

Executar

VOCÊ DIGITA

```
$ mvn -q compile exec:java -Dexec.mainClass=EnviarPaciente
```

VOCÊ VÊ

```
Comprovante: https://localhost:8443/Task/224d9f1d-1580-4007-a881-a0b11f88644d
Registro aceito: Patient/…
```

Para enviar o registro com erros, informe o arquivo na execução:

VOCÊ DIGITA

```
$ mvn -q compile exec:java -Dexec.mainClass=EnviarPaciente -Dexec.args=paciente-com-erros.json
```

VOCÊ VÊ

```
Comprovante: https://localhost:8443/Task/…
Registro recusado: Falha na validação HAPI FHIR
  error HAPI-1821: [element="gender"] Invalid attribute value "F": Unknown AdministrativeGender code 'F'
```

OBSERVE

A biblioteca oferece `PemLoader` e `SslContextFactory`, que montam a conexão a partir dos arquivos PEM. O programa usa o Jackson 3; versões do Jackson 2 anteriores à 2.22 entram em conflito com a biblioteca.

☐ **Passo concluído**

PARTE 3

Verificação

Responda sem consultar o texto. As respostas estão no final do módulo.

1. O envio retornou 202. O registro já foi aceito pelo HubSaúde?

2. A conexão caiu logo após o envio, e o sistema não recebeu resposta. O que fazer?

3. Uma repetição com a mesma chave recebeu 409. O registro foi recusado?

4. Por que um registro aceito pelo Simulador ainda pode ser recusado no ambiente oficial?

5. Onde está o motivo de uma recusa?

Glossário do módulo

202
Código de resposta que confirma o recebimento do envio, sem o resultado.

409
Resposta a uma repetição que chega enquanto o envio original ainda está em processamento.

Content-Location
Cabeçalho da resposta que traz o endereço da Task.

correlation-id
Identificador de um pedido, útil ao pedir ajuda ao suporte.

Idempotency-Key
Identificador único de um envio, que permite repeti-lo sem duplicar o registro.

OperationOutcome
Recurso FHIR que descreve problemas, um por item de `issue`.

Retry-After
Cabeçalho que sugere, em segundos, quanto tempo esperar antes de consultar ou repetir.

Task
Recurso FHIR que acompanha o processamento de um envio até o resultado.

PRÓXIMO MÓDULO

6. Projeto final

A integração completa, do registro ao resultado, montada por você a partir do que foi visto nos módulos anteriores.

Respostas da verificação

1. *O envio retornou 202. O registro já foi aceito pelo HubSaúde?*

Não necessariamente. O 202 confirma apenas o recebimento. A aceitação é conhecida ao consultar a Task: `completed` indica aceitação, e `rejected` indica recusa.

2. *A conexão caiu logo após o envio, e o sistema não recebeu resposta. O que fazer?*

Repetir o envio com a mesma `Idempotency-Key`. Se o registro já tiver chegado, o HubSaúde devolve o mesmo comprovante, sem duplicá-lo.

3. *Uma repetição com a mesma chave recebeu 409. O registro foi recusado?*

Não. O 409 indica que o envio original ainda está em processamento. Basta aguardar o tempo do `Retry-After` e repetir com a mesma chave, ou consultar a Task.

4. *Por que um registro aceito pelo Simulador ainda pode ser recusado no ambiente oficial?*

Porque o Simulador confere apenas a estrutura do registro, enquanto o ambiente oficial aplica as regras completas dos Guias de Implementação. A conferência no Validador antes do envio evita essa diferença.

5. *Onde está o motivo de uma recusa?*

No OperationOutcome incluído em `contained`, dentro da própria Task. O campo `statusReason` traz um resumo, e cada item de `issue` descreve um problema.